

Introducción a robustez y seguridad

En **Unidad 01** montaste un asistente: estado, configuración, `build_assistant_prompt`, llamadas a Gemini.

En **esta unidad** asumimos que **el usuario puede escribir cualquier cosa** — incluido texto que intenta **cambiar las instrucciones** del sistema.

No cambias de modelo. Refuerzas el sistema alrededor del modelo.

Objetivos de la unidad

Al terminar, deberías poder:

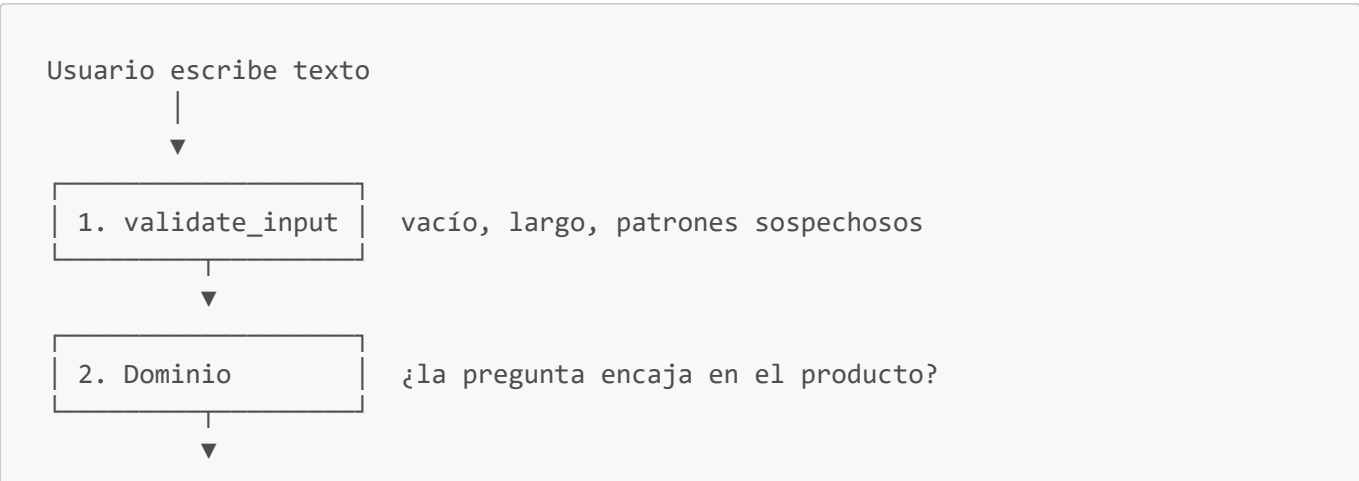
- Reconocer un **prompt injection** básico en entradas de usuario.
- Separar **instrucciones del sistema** del **input del usuario** en el código.
- **Acotar el dominio** del asistente (p. ej. solo tutor Python).
- Implementar `validate_input()` antes de llamar al LLM.
- Pedir **salida JSON**, parsearla y manejar errores en Python.
- Comparar comportamiento **vulnerable vs seguro** con el mismo input.

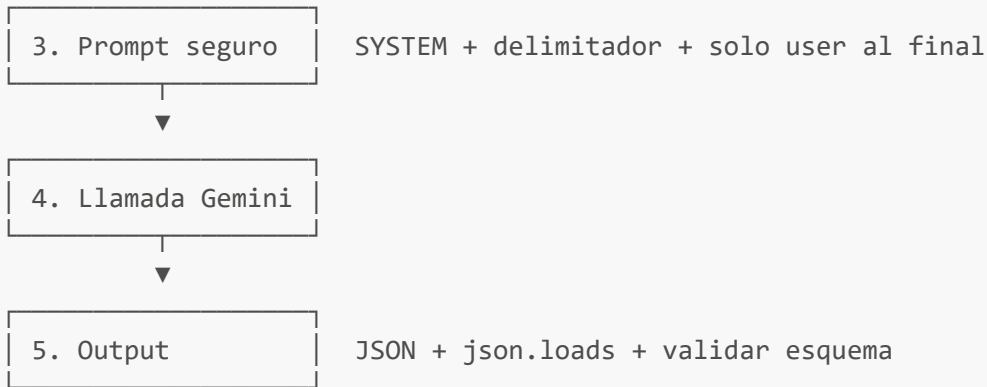
Qué es (y qué no es) robustez aquí

Sí es	No es
Capas en tu código Python	Garantía absoluta contra ataques avanzados
Validar, delimitar, rechazar antes del modelo	“Seguridad militar” o red team completo
Diseño consciente del canal usuario → prompt	Sustituir revisión humana en producción
Continuidad del asistente de U1	Framework de seguridad externo

En bootcamp el objetivo es **conciencia y patrones básicos**, no certificación de seguridad.

Defensa en capas (mapa mental)





Si falla la capa 1 o 2, **no llamas** al modelo (ahorras tokens y reduces superficie).

Convenciones de código

```
proyecto_seguro/  
├── config.py          # SYSTEM_PROMPT, dominio, límites  
├── validators.py      # validate_input()  
├── prompts.py         # build_vulnerable_prompt / build_secure_prompt  
├── logic.py           # turnos seguros vs vulnerables  
├── gemini_client.py  
└── main.py
```

Resumen en una frase

Robustez del asistente = validar y estructurar en Python antes y después del LLM, asumiendo que el usuario controla parte del texto que envías.